

Major Project

Design and Development of an Online Shopping Mobile Application



Anderson Eshop Report

Group Members:

- | | |
|--------------------|--------------|
| 1. Brendan MARAN | ID: 25530136 |
| 2. Deborah RAIMBAS | ID: 25530143 |
| 3. Valentino TOMON | ID: 25530376 |

Course: IS223

Lecturer: Raunu Gebo Sarsoruo

Date of submission: 9th October 2026

Application Version: 1.0

2. Abstract

This project presents the design and development of Anderson Eshop, a functional prototype of an online shopping mobile application developed using Java and Android Studio. The application was created for a hypothetical retail supermarket business operating in Lae City, Morobe Province, Papua New Guinea. Anderson Eshop allows customers to browse product categories (Bakery, Butchery, Drinks, Dairy, and Fruit & Vegetables / Produce), view product details and prices, search and filter products, add items to a shopping cart, calculate the total cost (including bulk discounts), and complete a simulated checkout process.

The project demonstrates the practical application of object-oriented programming concepts including classes, objects, constructors, methods, encapsulation, inheritance, method overriding, and polymorphism. An ArrayList collection is used to manage products in the shopping cart. The user interface was designed using Android XML layouts and integrated with Java logic through Activities and event handling.

This academic prototype does not process real financial transactions. The completed application (version 1.0) successfully meets the minimum functional requirements of the IS223 Major Project and provides a clear demonstration of how object-oriented programming principles can be applied to a real-world mobile information system.

3. Table of Contents

2. Abstract	1
4. Introduction	3
5. Problem Statement	3
6. Aim and Objectives.....	3
Aim	3
Objectives	4
7. Target Users	4
8. Requirements Analysis	4
8.1 Functional Requirements	4
8.2 Non-Functional Requirements	5
8.3 Hardware and Software Requirements	5
9. System Design.....	5
10. UML Diagrams	6
10.1 Use Case Diagram	6
10.2 Class Diagram.....	6
10.3 Activity / Navigation Diagram.....	7
11. Java OOP Implementation	8
11.1 Classes and Objects	8
11.2 Constructors	9
11.3 Inheritance	9
11.4 Method Overriding	9
11.5 Polymorphism.....	9
11.6 ArrayList Collection	9
12. Android Application Development	9
13. User Interface Design.....	10
14. Testing.....	10
15. Results.....	10
16. Challenges Encountered.....	11
17. Solutions.....	11
18. Conclusion	11
19. Recommendations	12
20. References.....	12
21. Appendices.....	12
Appendix A – GitHub Repository	12
Appendix B – Selected Source Code	12
Appendix C – Application Screenshots	18
Appendix D – Project Configuration	21

4. Introduction

The rapid growth of mobile technology has transformed the way people shop around the world. In Papua New Guinea, particularly in urban centres such as Lae City, many customers still rely on physical visits to retail stores and supermarkets to compare prices, check product availability, and make purchasing decisions. This traditional process can be time-consuming and inconvenient for busy customers.

The IS223 Major Project requires students to design and develop a functional online shopping mobile application using Java and Android Studio. The purpose of this project is to apply object-oriented programming concepts learned in class to a practical mobile application development problem.

This report documents the complete development of Anderson Eshop, an Android-based online shopping application created by a group of three students. The application enables users to browse supermarket products across multiple categories (Bakery, Butchery, Drinks, Dairy, and Fruit & Vegetables), manage a shopping cart, search and filter products, and complete a simulated order process. The project follows the recommended development process, including problem identification, requirements analysis, system design, Java OOP implementation, Android interface development, and testing.

By completing this project, the group has gained practical experience in combining Java object-oriented programming with Android mobile development to solve a realistic business problem.

5. Problem Statement

Many retail businesses and small supermarkets in Lae City, Morobe Province currently operate using traditional in-store shopping methods. Customers are required to physically visit the store to view available products, compare prices, and decide what to purchase. This process presents several challenges:

- Customers waste time travelling to and from the store.
- Product information and prices are not easily accessible outside store hours.
- It is difficult for customers to compare multiple products quickly.
- Businesses miss the opportunity to reach customers who prefer mobile shopping.

Anderson Eshop was developed to address these problems. The mobile application provides customers with a convenient platform to browse product categories (Bakery, Butchery, Drinks, Dairy, and Fruit & Vegetables), view product names and prices, search and filter items, select products, add them to a shopping cart, calculate the total cost (including special bulk discounts for bakery items), and complete a basic checkout process using their smartphone.

The application is designed as an academic prototype and does not process real payments. Its main purpose is to demonstrate how information technology and object-oriented programming can support the online shopping process for a retail business in Papua New Guinea.

6. Aim and Objectives

Aim

The aim of this project is to design and develop a functional Android-based online shopping mobile application named Anderson Eshop using Java and Android Studio that demonstrates the application of object-oriented programming concepts to a real-world supermarket business problem in Lae City, Papua New Guinea.

Objectives

The specific objectives of this project are to:

1. Analyse the requirements of an online shopping application for a retail supermarket business.
2. Identify a suitable business context (Anderson Eshop) for the application.
3. Design an appropriate mobile application interface.
4. Apply Java object-oriented programming principles in the development of the application.
5. Implement classes and objects to represent products, customers, shopping carts, and orders.
6. Apply constructors and methods to initialise and manipulate application objects.
7. Demonstrate inheritance using appropriate parent and child classes.
8. Demonstrate method overriding where appropriate.
9. Apply polymorphism in the application.
10. Implement a functional shopping cart.
11. Calculate product subtotals and the overall shopping total.
12. Develop an Android graphical user interface using XML layouts.
13. Implement event handling using Java.
14. Test the application and identify programming or usability errors.
15. Document the design, implementation, testing, and evaluation of the application.

7. Target Users

The primary target users of Anderson Eshop are:

- Residents of Lae City and surrounding areas in Morobe Province who prefer convenient mobile shopping for everyday supermarket items.
- Working adults, students, and families who have limited time to visit physical stores.
- Customers who want to browse bakery products, meat, drinks, dairy, and fresh fruit & vegetables, compare prices, and prepare orders using their smartphones.
- People with basic smartphone knowledge who can navigate simple mobile applications.

Secondary users include the business owners or staff of Anderson Eshop who may later expand the application for order management.

8. Requirements Analysis

8.1 Functional Requirements

The Anderson Eshop application must allow users to:

- Open the application and view a home screen with the business name and navigation options.
- Browse product categories (Bakery, Butchery, Drinks, Dairy, and Fruit & Vegetables / Produce).
- View a list of products showing name, price, description, and category.
- Search products by name or description in real time.
- Filter products by category chips and by minimum/maximum price.
- Select a product and add it to the shopping cart (with quantity controls).
- View the shopping cart with selected products, quantities, prices, and total cost.
- Enter basic customer information (name, telephone number, and delivery location).

- Review the order and complete a simulated checkout process.
- Receive an order confirmation message.
- Optionally switch between light and dark theme on the home screen.

8.2 Non-Functional Requirements

- The application must have a clear, consistent, and user-friendly interface.
- Navigation between screens must be simple and logical.
- The application should run smoothly on standard Android devices and emulators (minimum API 24).
- Text and buttons must be readable on mobile screens.
- The shopping cart total must calculate correctly (including bulk discounts where applicable).
- The application is a prototype and must clearly indicate that it does not process real payments.

8.3 Hardware and Software Requirements

- Android Studio development environment
- Java programming language
- Android SDK and emulator (or physical Android device)
- Minimum Android version: API 24 (Android 7.0); target API 34
- Application version: 1.0

9. System Design

Anderson Eshop was designed with a clear separation between the user interface and the application logic. The system architecture consists of:

- User Interface Layer: Android Activities and XML layout files that present information to the user.
- Business Logic Layer: Java classes that handle products, shopping cart operations, and order processing.
- Shared Data Layer: A simple static class (ShopData) that holds the shopping cart so that items are not lost when moving between screens.

The main flow of the application is:

Home Screen → Categories / Products → Product List (search & filter) → Add to Cart → View Cart → Checkout → Order Confirmation

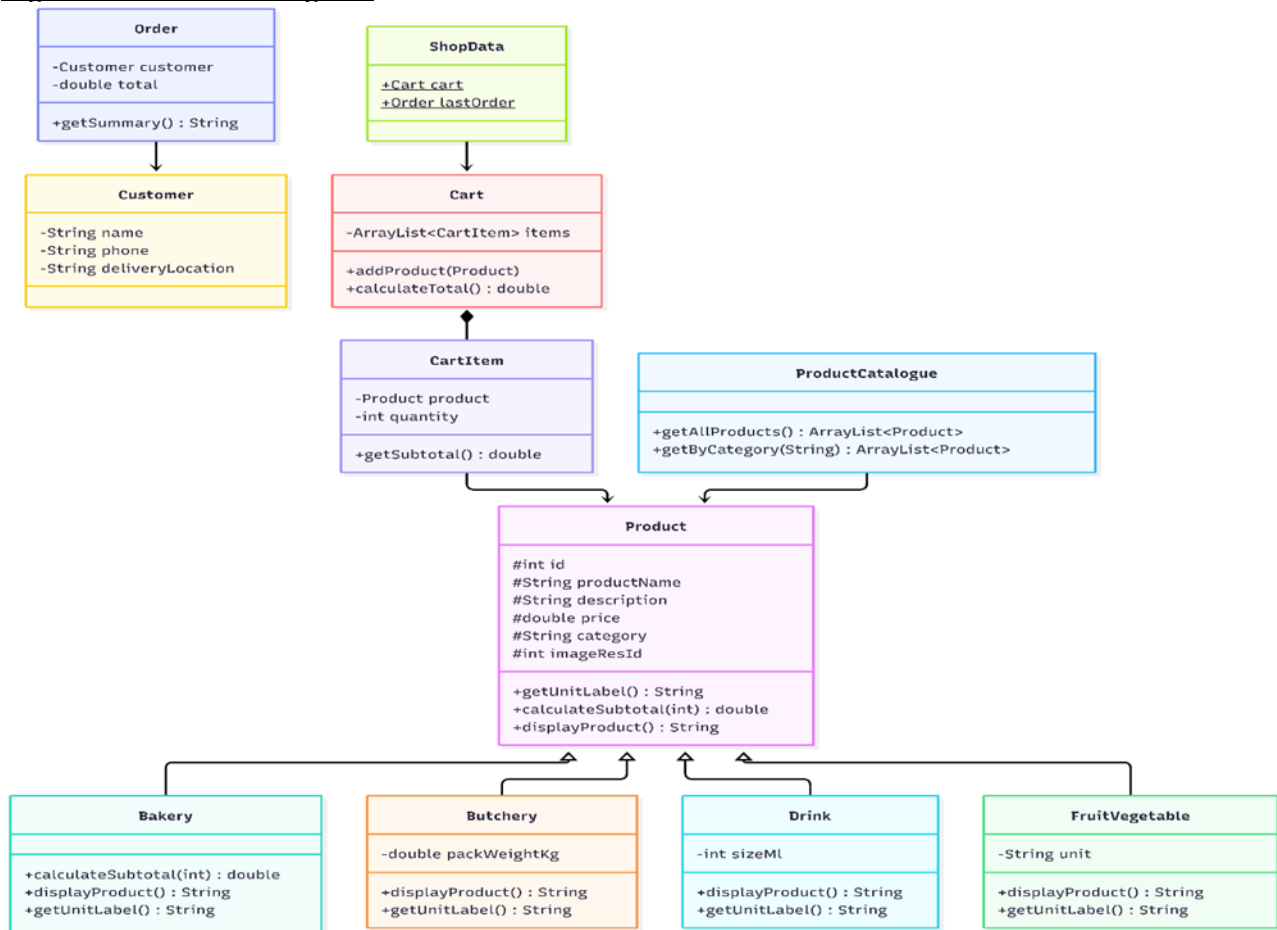
This modular design makes the application easy to understand, test, and maintain.

10. UML Diagrams

The following diagrams support the design of Anderson Eshop:

10.1 Use Case Diagram

Figure 1. Use Case Diagram



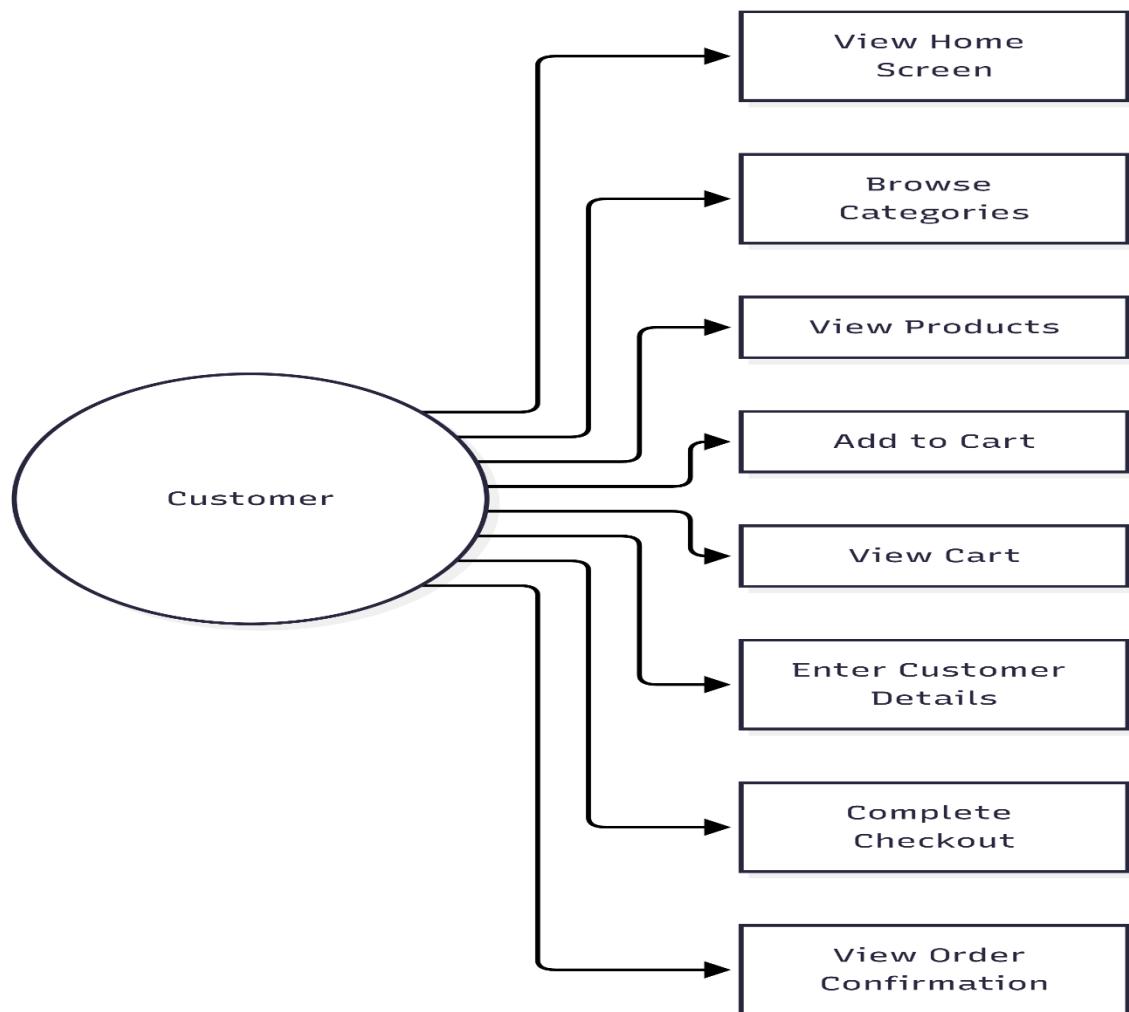
The Use Case Diagram shows the main interactions between the Customer and the system. The primary use cases are: View Home Screen, Browse Categories, View Products, Search/Filter Products, Add Product to Cart, View Cart, Enter Customer Details, and Complete Checkout.

10.2 Class Diagram

The Class Diagram shows the object-oriented structure of the application. The main classes are:

- Product (parent class)
- Bakery, Butchery, Drink, and FruitVegetable (child classes)
- Cart and CartItem
- Customer
- Order
- ShopData (holds the shared cart)
- ProductCatalogue (holds the product list)

Figure 2. Class Diagram of Anderson Eshop

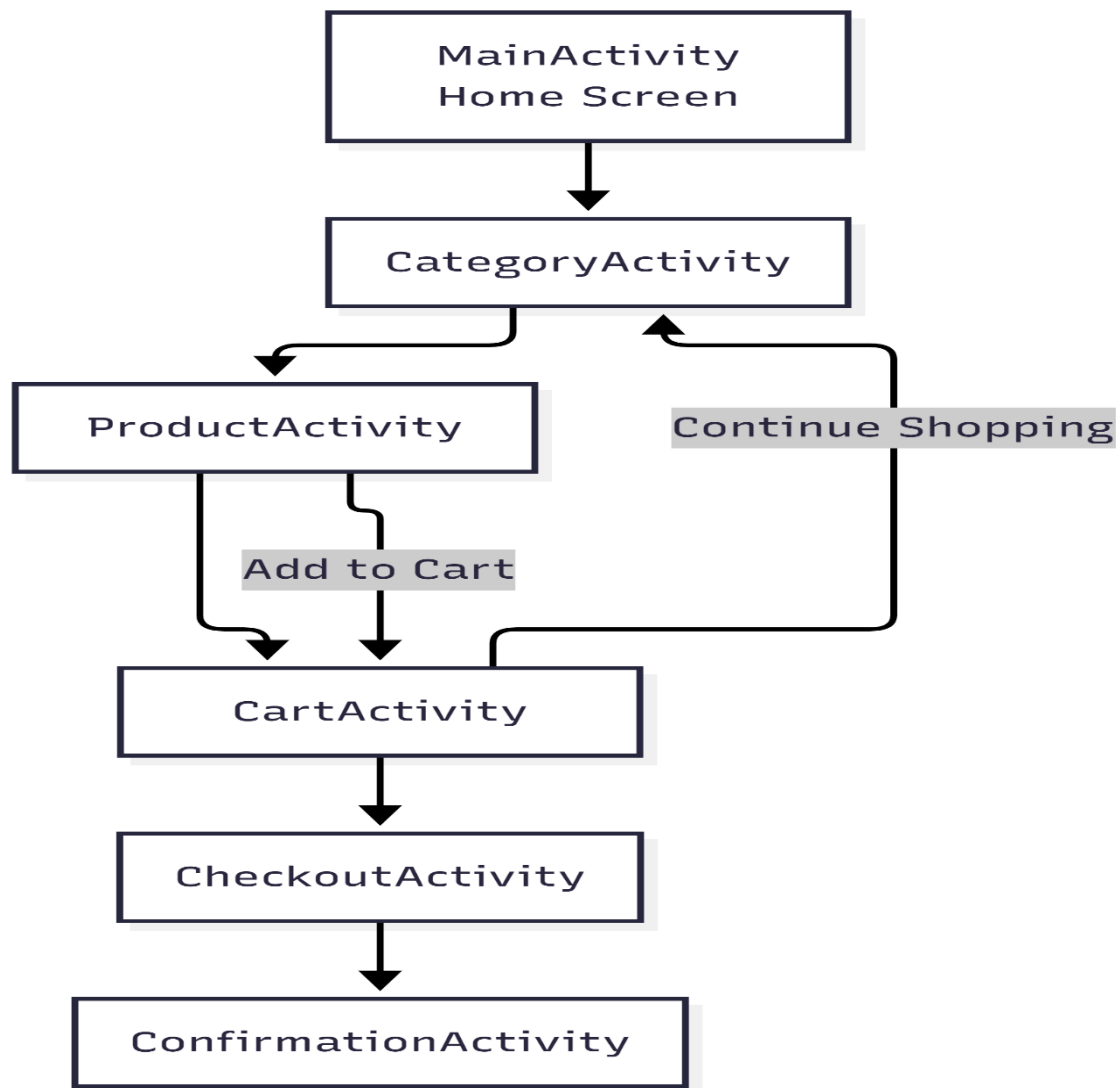


The diagram clearly illustrates the inheritance relationship between Product and its subclasses, as well as the composition relationship between Cart and CartItem.

10.3 Activity / Navigation Diagram

This diagram shows the sequence of screens a user follows when using the application, starting from the main screen and ending at the order confirmation screen.

Figure 3. Activity / Navigation flow Chart



11. Java OOP Implementation

Object-oriented programming principles were applied throughout the development of Anderson Eshop. The implementation is described below based on the actual source code.

11.1 Classes and Objects

The main classes created are:

- **Product** – the parent class containing common attributes (id, productName, description, price, category, imageResId).
- **Bakery, Butchery, Drink, and FruitVegetable** – specialised child classes.
- **Cart** – manages the collection of selected products.
- **CartItem** – represents a product and its quantity in the cart.
- **Customer** – stores customer name, phone number, and delivery location.
- **Order** – represents a completed order.
- **ShopData** – provides a shared cart instance across all activities.
- **ProductCatalogue** – static catalogue of all products available in the shop.

11.2 Constructors

Constructors are used to initialise objects with the required values. The Product constructor accepts id, name, description, price, category, and image resource. Child class constructors call the parent constructor using `super(...)`.

11.3 Inheritance

Inheritance is demonstrated by creating a parent class Product and four child classes (Bakery, Butchery, Drink, FruitVegetable). The child classes inherit common attributes and methods and specialise behaviour where needed. Dairy items are represented using the Product class with category "Dairy".

11.4 Method Overriding

Method overriding is implemented in the child classes. Examples include:

- `displayProduct()` – each child class provides a specialised display format.
- `getUnitLabel()` – returns category-specific units (e.g. "item", "kg", "pack").
- `calculateSubtotal(int quantity)` – the Bakery class overrides this method to apply a 10% bulk discount when 12 or more items are purchased.

11.5 Polymorphism

Polymorphism is applied by treating objects of different child classes as type Product. This allows the shopping cart and product lists to handle any product type through a common interface. The correct overridden method is executed at runtime depending on the actual object type.

11.6 ArrayList Collection

The Cart class uses an `ArrayList<CartItem>` to store the products selected by the customer. Methods such as `addProduct()`, `removeProduct()`, and `calculateTotal()` operate on this collection. The shared cart is accessed through the ShopData class so that cart contents persist across different screens. ProductCatalogue also uses an `ArrayList<Product>` to hold the full product range.

These object-oriented features form the core logic of Anderson Eshop and fully meet the programming requirements of the IS223 Major Project.

12. Android Application Development

The Android application was developed using Android Studio. The main Activities implemented are:

- MainActivity – Home screen of the application (includes dark theme toggle)
- CategoryActivity – Displays product categories
- ProductActivity – Shows the product list with search, category chips, and price filtering
- CartActivity – Displays selected items and the total cost
- CheckoutActivity – Collects customer details
- ConfirmationActivity – Shows the final order confirmation

XML layout files were created for each screen (`activity_main.xml`, `activity_category.xml`, `activity_product.xml`, `activity_cart.xml`, `activity_checkout.xml`, and `activity_confirmation.xml`). Additional item layouts (`item_product.xml` and `item_cart.xml`) are used for displaying individual products and cart items.

Java event listeners are used to handle button clicks and navigation between screens using Intents. The shared cart from ShopData allows products added on the product screen to remain available on the cart and checkout screens. The application targets API 34 with a minimum of API 24 and is released as version 1.0.

13. User Interface Design

The user interface of Anderson Eshop was designed to be simple, clear, and suitable for mobile devices. Key design decisions include:

- Consistent layout structure across all screens
- Clear navigation buttons on the home screen (View Products, Categories, Shopping Cart)
- Readable product names, descriptions, and prices
- Product images (or colour placeholders where full photos were not available)
- Category filter chips and real-time search on the product screen
- Min/max price filter for product discovery
- A clean shopping cart display showing quantity and subtotal with +/- controls
- A simple checkout form for customer name, phone number, and delivery location
- Optional dark theme toggle on the home screen

The interface follows basic mobile design principles to ensure usability for the target users in Lae City.

14. Testing

The application was tested using the Android emulator and a physical Android device. A testing table was prepared to check the main functions of the system.

Table 1. Testing Results

No.	Test Case	Expected Result	Status
1	Launch application	Home screen appears	Pass
2	Select a category	Product list for that category opens	Pass
3	Search products by name	Matching products are listed	Pass
4	Filter by price range	Only products within range shown	Pass
5	Add product to cart	Product is added successfully	Pass
6	View shopping cart	Selected products and total are shown	Pass
7	Increase/decrease quantity	Quantity and total update correctly	Pass
8	Proceed to checkout	Checkout screen appears	Pass
9	Enter customer details and submit	Order confirmation is displayed	Pass
10	Bakery bulk discount (12+ items)	10% discount applied correctly	Pass
11	Run on physical Android device	Application installs and runs successfully	Pass

Functional testing, navigation testing, and calculation testing were performed. Minor issues found during testing were corrected before final submission.

15. Results

The completed Anderson Eshop application (version 1.0) successfully meets the minimum functional requirements of the IS223 Major Project. Users can:

- Browse product categories (Bakery, Butchery, Drinks, Dairy, Fruit & Vegetables / Produce)
- Search products by name or description
- Filter products by category chips and by price range
- View product details and prices
- Add products to a shopping cart and adjust quantities
- View and manage cart items
- Calculate the correct total (including bulk discount for bakery items)
- Enter customer information
- Complete a simulated checkout process
- Toggle dark theme on the home screen

The application also successfully demonstrates inheritance, method overriding, polymorphism, and the use of ArrayList. It was verified on both the Android emulator and a physical Android device.

16. Challenges Encountered

During development the group faced several challenges:

- Connecting the Java classes correctly to the Android Activities
- Ensuring the shopping cart retained items when moving between screens
- Implementing method overriding and polymorphism correctly
- Designing clean and consistent XML layouts
- Configuring the incomplete GitHub project so it would build and run in Android Studio (Gradle / JDK compatibility)
- Managing time for testing and documentation

17. Solutions

The challenges were solved by:

- Creating the ShopData class to hold a shared cart instance
- Testing Java classes independently before integrating them with the interface
- Using the Android Studio debugger and Logcat to identify problems
- Adding the missing Gradle project files and aligning the Gradle JDK so the project could sync and run
- Regular group meetings and GitHub commits to keep the project organised
- Following the recommended development roadmap provided in the project guide

18. Conclusion

Anderson Eshop is a functional prototype of an online shopping mobile application developed using Java and Android Studio. The project successfully demonstrates the application of object-oriented programming concepts (classes, objects, constructors, inheritance, method overriding, polymorphism, and ArrayList) to a realistic business problem.

By completing this project, the group has gained practical experience in designing, implementing, and testing a complete mobile information system. The final application (version 1.0) is usable on emulator and physical devices, well-structured, and meets the requirements of the IS223 Major Project.

19. Recommendations

Future improvements to Anderson Eshop could include:

- User login and registration
- Persistent storage using a local database or Firebase
- Order history for customers
- Product ratings and reviews
- Full product photography set (replacing placeholder images)
- Real payment gateway integration (for a production version)

Search, category filtering, and price filtering are already implemented in the current prototype and are not listed as future work. The features above were not required for the current academic prototype.

20. References

- Official Android Developer Documentation. Available at: <https://developer.android.com>
- IS223 Major Project Description Guide (Raunu Gebo Sarsoruo, 2026)
- Java Object-Oriented Programming textbooks and resources used during the course

21. Appendices

Appendix A – GitHub Repository

Repository URL: <https://github.com/Br737-code/Anderson Eshop>

Appendix B – Selected Source Code

B.1 Product.java (Parent Class)

```

package com.example.andersoneshop;

/**
 * Parent class for every item sold at Anderson's Shop.
 */
public class Product {

    // Product attributes
    protected int id;
    protected String productName;
    protected String description;
    protected double price;
    protected String category;
    protected int imageResId;

    // Constructor
    public Product(int id, String productName, String description,
                  double price, String category, int imageResId) {

        this.id = id;
        this.productName = productName;
        this.description = description;
        this.price = price;
        this.category = category;
        this.imageResId = imageResId;
    }

    // Getter methods
    public int getId() {
        return id;
    }

    public String getProductName() {
        return productName;
    }

    public String getDescription() {
        return description;
    }

    public double getPrice() {
        return price;
    }

    public String getCategory() {
        return category;
    }

    public int getImageResId() {
        return imageResId;
    }

    // Returns the default unit label
    public String getUnitLabel() {
        return "each";
    }

    // Calculates the subtotal based on quantity
    public double calculateSubtotal(int quantity) {
        return price * quantity;
    }

    // Displays the product name and formatted price
    public String displayProduct() {
        return productName + " - K" + String.format("%.2f", price);
    }
}

```

B.2 Bakery.java (Child Class – shows Inheritance + Method Overriding)

```

package com.example.andersoneshop;

public class Bakery extends Product {

    // Bulk purchase constants
    private static final int BULK_QUANTITY = 12;
    private static final double BULK_DISCOUNT = 0.10; // 10% off when buying 12 or more

    // Constructor
    public Bakery(int id, String productName, String description,
                  double price, int imageResId) {

        super(id, productName, description, price, "Bakery", imageResId);
    }

    // Overridden method: returns the unit label
    @Override
    public String getUnitLabel() {
        return "item";
    }

    // Overridden method: applies bulk discount
    @Override
    public double calculateSubtotal(int quantity) {

        double subtotal = super.calculateSubtotal(quantity);

        if (quantity >= BULK_QUANTITY) {
            subtotal = subtotal * (1 - BULK_DISCOUNT);
        }

        return subtotal;
    }

    // Overridden method: displays bakery-specific product information
    @Override
    public String displayProduct() {

        return "Bakery: " + productName
            + " - K" + String.format("%.2f", price)
            + " (baked fresh daily)";
    }
}

```

B.3 Cart.java

```

package com.example.andersoneshop;

import java.util.ArrayList;

public class Cart {

    // Stores all items currently in the cart
    private ArrayList<CartItem> items = new ArrayList<>();

    // Adds a product to the cart
    public void addProduct(Product product) {

        for (CartItem item : items) {

            // If the product already exists, increase its quantity
            if (item.getProduct().getId() == product.getId()) {
                item.setQuantity(item.getQuantity() + 1);
                return;
            }
        }

        // Add the product as a new cart item
        items.add(new CartItem(product, 1));
    }

    // Removes one quantity of a product from the cart
    public void removeProduct(Product product) {

        for (int i = 0; i < items.size(); i++) {

            CartItem item = items.get(i);

            if (item.getProduct().getId() == product.getId()) {

                if (item.getQuantity() > 1) {
                    item.setQuantity(item.getQuantity() - 1);
                } else {
                    items.remove(i);
                }

                return;
            }
        }
    }

    // Calculates the total cost of all items
    public double calculateTotal() {

        double total = 0;

        for (CartItem item : items) {
            total += item.getSubtotal();
        }

        return total;
    }

    // Returns all items in the cart
    public ArrayList<CartItem> getItems() {
        return items;
    }

    // Checks whether the cart is empty
    public boolean isEmpty() {
        return items.isEmpty();
    }

    // Removes all items from the cart
    public void clear() {
        items.clear();
    }
}

```


B.4 CartItem.java (shows Polymorphism)

```
package com.example.andersoneshop;

public class CartItem {

    // Stores the product and its quantity
    private Product product;
    private int quantity;

    // Constructor
    public CartItem(Product product, int quantity) {
        this.product = product;
        this.quantity = quantity;
    }

    // Getter method for the product
    public Product getProduct() {
        return product;
    }

    // Getter method for the quantity
    public int getQuantity() {
        return quantity;
    }

    // Updates the quantity of the product
    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

    /**
     * Polymorphism: the correct calculateSubtotal()
     * method runs for each product type.
     */
    public double getSubtotal() {
        return product.calculateSubtotal(quantity);
    }
}
```

B.5 ShopData.java

```

package com.example.andersoneshop;

/**
 * Shared application data.
 * Every screen uses the same cart, so items are not lost
 * when navigating between screens.
 */
public class ShopData {

    // Shared shopping cart used across all screens
    public static Cart cart = new Cart();

    // Stores the most recently completed order
    public static Order lastOrder = null;
}

```

B.6 FruitVegetable.java (another Child Class example)

```

package com.example.andersoneshop;

public class FruitVegetable extends Product {

    // Unit of measurement, e.g. "kg", "bunch", or "each"
    private String unit;

    // Constructor
    public FruitVegetable(int id, String productName, String description,
                        double price, String unit, int imageResId) {

        super(id, productName, description, price,
            "Fruit & Vegetables", imageResId);

        this.unit = unit;
    }

    // Overridden method: returns the product's unit
    @Override
    public String getUnitLabel() {
        return unit;
    }

    // Overridden method: displays fresh produce information
    @Override
    public String displayProduct() {

        return "Fresh: " + productName
            + " - K" + String.format("%.2f", price)
            + " per " + unit;
    }
}

```

Appendix C – Application Screenshots

Figure C1: Home Screen (MainActivity)

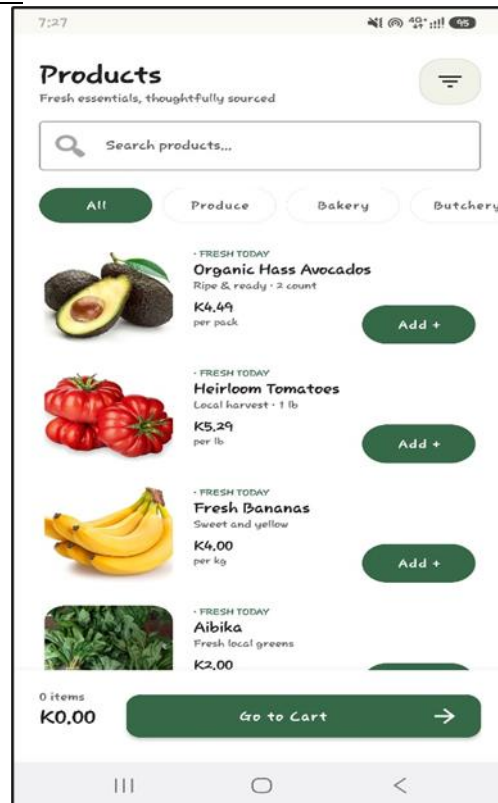


Description: The home screen displays the application name, welcome message, and navigation buttons to Products, Categories, Shopping Cart, and the Dark Mode toggle.

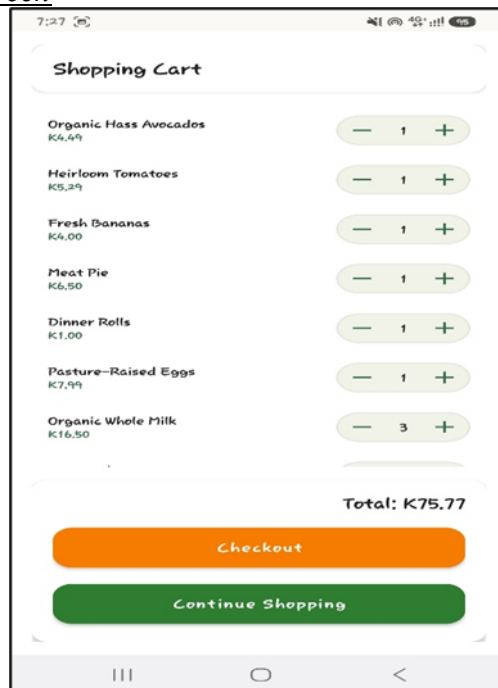
Figure C2: Category Screen



Description: Shows the available product categories (Bakery, Butchery, Drinks, Dairy, Fruit & Vegetables).

Figure C3: Product List Screen

Description: Displays the list of products with search bar, category chips, and price filter options.

Figure C4: Shopping Cart Screen

Description: Shows selected products, quantity controls (+/-), individual subtotals, and the overall total.

Figure C5: Checkout Screen

7:27

Checkout

PROTOTYPE ONLY: this is a school project. No real payment is processed.

Delivery Details

Full Name

Phone Number

Delivery Location (e.g. Eriku, Lae)

Order Review

- 1 x Organic Hass Avocados K4,49
- 1 x Heirloom Tomatoes K5,29
- 1 x Fresh Bananas K4,00
- 1 x Meat Pie K6,50
- 1 x Dinner Rolls K1,00
- 1 x Pasture-Raised Eggs K7,99

Place Order

Description: Customer enters Name, Phone Number, and Delivery Location before confirming the order.

Figure C6: Order Confirmation Screen

7:28

Order Confirmed!

Thank you for shopping at Anderson's

Order #1001
Customer: Brendan
Phone: 71314589
Deliver to: Unitech

- 1 x Organic Hass Avocados K4,49
- 1 x Heirloom Tomatoes K5,29
- 1 x Fresh Bananas K4,00
- 1 x Meat Pie K6,50
- 1 x Dinner Rolls K1,00
- 1 x Pasture-Raised Eggs K7,99
- 3 x Organic Whole Milk K16,50
- 2 x Coca-Cola K9,00
- 3 x Orange Juice K21,00

TOTAL: K75,77
(Prototype only – no real payment processed)

Back to Home

Description: Displays the order summary and confirmation message after successful checkout.

Appendix D – Project Configuration

- **Application ID:** com.example.andersoneshop
- **Version:** 1.0 (versionCode 1)
- **Minimum SDK:** 24
- **Target / Compile SDK:** 34